Rog-Beep Controller for Arduino Nano — Manual & Wiring Diagrams

- 1. How it works
  - Timeline
  - The button on D3 — two functions
  - The switch
- 2. Parts
- 3. Wiring diagrams
  - 3.1 PTT route (the switch)
  - 3.2 PTT output to the radio
  - 3.3 Beep injection into the microphone audio line
  - 3.4 Power supply
- 4. Nano notes: the differences with the UNO
  - 4.1 Choosing board and processor in the IDE
  - 4.2 After uploading: press RESET
  - 4.3 USB chip and drivers
  - 4.4 A6 and A7 are not digital pins
  - 4.5 The onboard LED on D13 is dim
  - 4.6 Power supply and current
  - 4.7 A free second UART on D14/D15
- 5. Pin overview
- 6. Wiring it up — step by step
  - Test measurements
- 7. Software settings
  - 7.1 Directly in the sketch (compile time)
  - 7.2 While in use (serial monitor)
- 8. Setting the level
- 9. Troubleshooting
- 10. Possible extensions
- 11. Practical notes
- 12. The tests (optional)
- 13. Quick reference

Project: put an Arduino **Nano** between your microphone (PTT) and your transceiver so it takes care of the *PTT tail* (roger beep): when you release the PTT the radio keeps transmitting for a moment and you hear one or more beeps.

File	Contents
`rogbeep.ino`	the Arduino sketch (the only file you open in the IDE)
`MANUAL.md`	this document
`MANUAL.pdf`	the same manual, print friendly (A4, with table of contents)
`test/`	tests of the logic (optional, see §12)

> **Why the Nano?** The Nano has the same processor as the UNO (ATmega328P, 16 MHz) and the same pin numbering, so the sketch runs on both without changes. There are a few differences to keep in mind: **A6 and A7 are the I2C pins on the Nano and cannot be used as digital I/O** (that is why this project uses D2/D3/D8/D9), the **onboard LED on D13 is dim**, and uploading works differently. See §4.

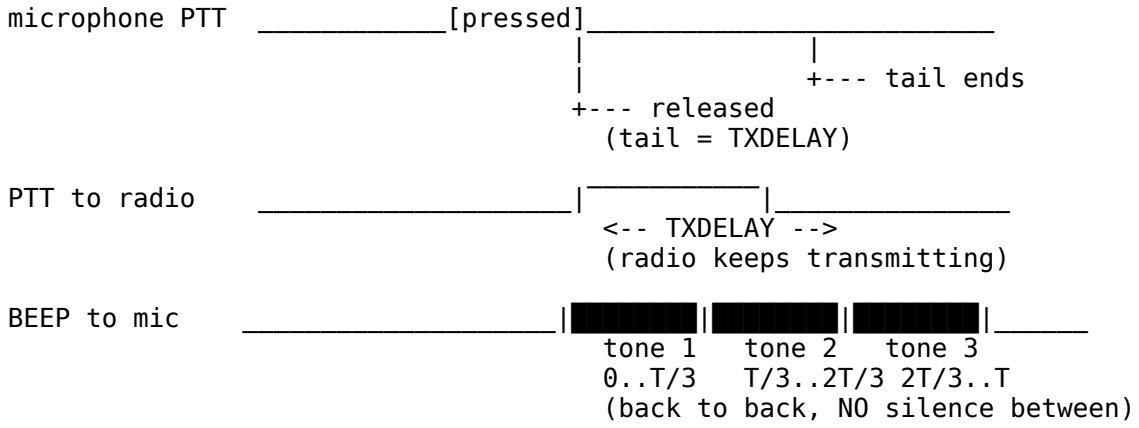
---

1. How it works

The Nano sits *inline* between microphone and transceiver and has four connections:

```
INPUT   PTT   (from the microphone)
OUTPUT  PTT   (to the radio)
OUTPUT  BEEP  (to the microphone audio line)
SETTINGS  TXDELAY (0..1000 ms), number of beeps (1..3), frequencies (700..2000 Hz)
```

Timeline



- **Pressed** — the PTT passes through to the radio almost immediately (max. ~15 ms debounce). There is **no** beep while you are transmitting.
- **Released** — the Nano keeps the PTT asserted for another `TXDELAY` ms. During that tail the tone pattern plays: 1, 2 or 3 tones, each with its own frequency (700–2000 Hz).
- **Tail over** — tone off, PTT released, the radio stops transmitting.

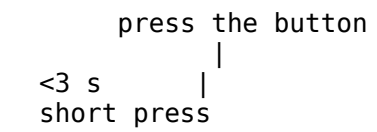
Extras the sketch already handles:

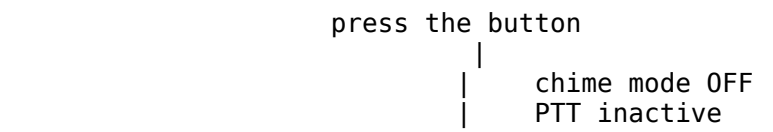
- **Pressing again during the tail** → the tone stops immediately and you keep transmitting; a new tail is *not* started.
- **TXDELAY = 0** → the radio stops as soon as you release the PTT (no tail).
- **Button** → a short press plays the tone pattern once *without* transmitting (handy for setting the volume).

The button on D3 — two functions

The button has a **short press** and a **long press**:

Action	Behaviour
Short press (less than 3 s)	The tone pattern plays once, **without** the radio transmitting. Useful for setting the level with a monitor receiver or the radio's MIC gain.
Long press (hold 3 s)	**Chime mode** switches on: the tone pattern repeats endlessly and the PTT becomes **active**.
Hold 3 s again	Chime mode switches off: the tone stops and the PTT becomes **inactive**.





```
          v                                      v
pattern plays once, no TX     chime mode ON  |   silence
(check the level)              PTT ACTIVE     |
                               chime repeats  |
                               endlessly ------+
```

Practical notes:

- **Chime mode stays on** after you release the button — it keeps running until you hold the button down for 3 seconds again. Handy to let the chime sound while you adjust the radio, or to record a beacon.
- The long press fires **while you are still holding the button** (after 3 s). You do not have to release it to hear the effect.
- In chime mode the **microphone PTT is ignored**: chime mode holds the PTT itself. As soon as you switch chime mode off, the microphone PTT does its normal job again.
- The chime has **its own cycle length of 1800 ms** and **always plays three tones** — both independent of the TXDELAY setting and of the `beeps` setting. Three tones over 1800 ms is 600 ms per tone, back to back; the next cycle starts right after. So the chime sounds exactly the same every time, whatever you have changed elsewhere.
- The D13 LED is lit while the chime is playing (the PTT is active then).

The switch

A **mechanical switch** (SPDT, “single pole double throw”) decides who passes the PTT on:

Position	Route	Behaviour
DIRECT	microphone PTT → straight to the radio	The Nano does nothing. The radio works normally, even when the Nano is off or broken.
NANO	microphone PTT → Nano (D2); Nano (D8) → radio	The Nano handles the tail and the beep.

> **Note:** in the *NANO* position the radio **does not transmit** when the Nano is off or hung up. The Nano is “fail-safe”: it only pulls the PTT when told to. If you want to transmit with a dead Nano, put the switch in *DIRECT*.

---

2. Parts

Ref	Value	Function
—	Arduino Nano (ATmega328P)	the brain
Q1	BC547B / 2N3904 / 2N2222 (NPN)	switches the radio’s PTT input to GND
R1	10 kΩ	base resistor for Q1 (from D8)
R2	10 kΩ	protection of the Nano PTT input (from microphone PTT to D2)
R3	4.7 kΩ	limits the beep injection
RV1	10 kΩ pot (linear)	beep volume
C1	4.7 µF (electrolytic is fine, + to D9)	couples the tone to the audio line
C2	100 nF *(optional)*	across R3 → softer, less “harsh” tone

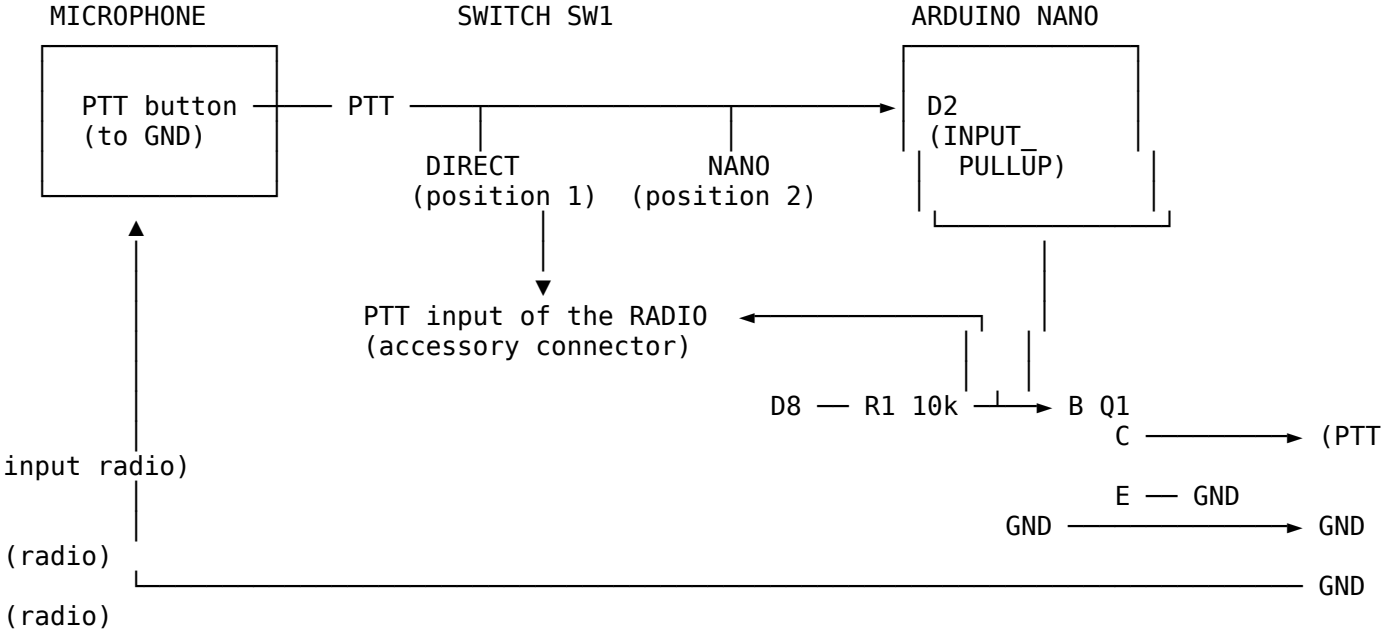
Ref	Value	Function
C3	100 nF + 10 µF *(optional)*	decoupling at the beep injection point
SW1	SPDT switch (or “on-off-on”)	switches DIRECT ↔ NANO
SW2	push button *(optional)*	button on D3: short = preview, long = chime mode
LED2	2 mA LED + 330 Ω *(optional)*	clear TX indicator on D13 (the onboard LED is dim)

Plus: solder, wire, a small enclosure if you like, and a **5 V supply** (USB from the PC, or a power supply on the barrel jack).

---

3. Wiring diagrams

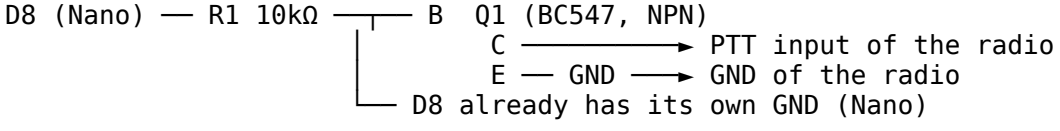
3.1 PTT route (the switch)



The microphone PTT is almost always a **contact that closes to GND**. Check that with a multimeter (continuity between the PTT conductor and the microphone GND while you press the button). The 10 kΩ (R2) between the PTT line and D2 is protection: should a higher voltage ever appear on that line, D2 cannot be damaged.

> The Nano GND and the radio GND are connected through the **microphone cable**. That is also the shortest route and keeps the noise level low.

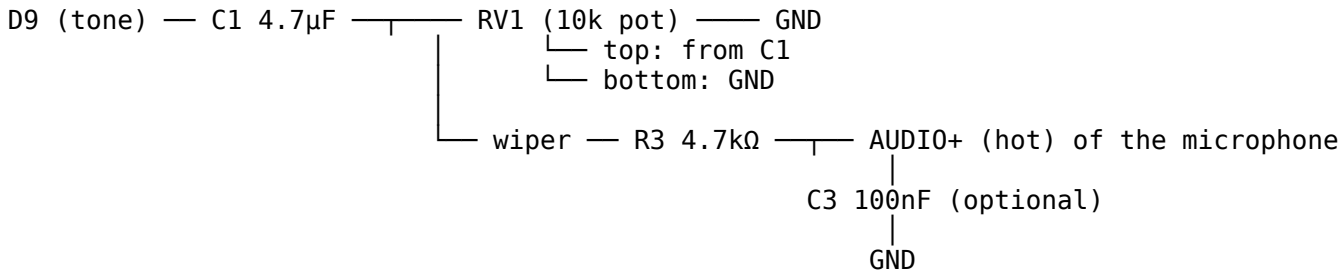
3.2 PTT output to the radio



Q1 works as an **open collector**: it connects the radio’s PTT input to GND, exactly like the microphone PTT button does. The radio therefore “sees” a PTT switch and does not know an Arduino is in the way. No voltage from the Nano reaches the radio’s PTT line, so the radio stays independent of the Nano.

Wiring D8 straight to the PTT input works on many radios too, but the transistor is safer: it galvanically isolates the Nano from the radio and mimics the switch exactly.

3.3 Beep injection into the microphone audio line



- C1 couples the **AC** of the tone through and blocks the DC. With an electrolytic: the **plus terminal towards D9**.
- RV1 is wired as a **volume control**: the wiper delivers the tone. The usable range is in the **first half** of the rotation, so start at minimum and turn it up carefully until the tone is clearly audible.
- R3 (4.7 kΩ) limits the current into the microphone line and keeps the level modest. Better too soft than too loud: a hard beep makes the transmitter “pump”.
- The AUDIO- (cold) conductor of the microphone is left alone; it goes straight to the radio’s AUDIO-.
- If your microphone has a single audio conductor (2-wire microphone), it is the same wire.

3.4 Power supply

- **USB** (laptop or USB power supply): the simplest option.
- **Barrel jack**: 7–12 V DC, the Nano regulates it down to 5 V.
- **From the radio (13.8 V)**: use a DC-DC converter (buck) 13.8 V → 5 V, 1 A minimum. Note that noise on the 13.8 V rail can be audible in the microphone. In that case use a separate 5 V supply or a battery.

  Tip against noise: fit C3 (100 nF + 10 µF) at the beep injection point and connect the Nano GND at a single point (the GND of the microphone cable).

---

4. Nano notes: the differences with the UNO

The Nano is electrically very similar to the UNO, but there are a few things that can trip you up.

4.1 Choosing board and processor in the IDE

Your board	Board in the IDE	Processor
Classic Nano (ATmega328P, CH340 or FT232)	**Arduino Nano**	**ATmega328P (Old Bootloader)** — usually the default
Original Nano from 2013 or older	Arduino Nano	ATmega328P (Old Bootloader)
Newer “Nano” with USB-C (often a CH340 clone)	Arduino Nano	ATmega328P (Old Bootloader)
Arduino Nano Every (ATmega4809)	**Arduino Nano Every**	— (different core, but the same pins)

This sketch only uses D2, D3, D8, D9 and D13, so it works on both variants.

4.2 After uploading: press RESET

The classic Nano has a bootloader that does **not** jump into the sketch by itself. After “Done uploading” you have to **press the RESET button once** (the small button next to the USB connector). If it looks like the sketch never started: upload again while holding RESET and release it as soon as the IDE reports the upload is done.

Some clones do have auto-reset; then you do not have to do anything.

4.3 USB chip and drivers

Many Nano clones use a **CH340G** instead of the original FT232. On Windows you then need to install the CH340 driver, otherwise the Nano does not show up in the IDE (it does show up in Device Manager as an “unknown device”). On Linux and macOS the driver is already part of the system.

4.4 A6 and A7 are not digital pins

On the Nano, A6 and A7 are the I2C pins (SDA/SCL) and **cannot** be used as digital inputs or outputs. This project therefore uses D2, D3, D8, D9 and D13 — all ordinary digital pins. If you later add an extra switch or potentiometer, use A0–A5 (as analog inputs) or D0–D13.

4.5 The onboard LED on D13 is dim

The LED on D13 of the Nano is a 2 mA type and is hard to see with the naked eye. For a clear TX indicator, fit your own LED (2 mA) with a 330 Ω resistor on D13. The sketch drives D13 high while transmitting and during the tail.

4.6 Power supply and current

- The Nano’s **5V pin** may supply at most ~500 mA, **including** the Nano’s own consumption. Our circuit asks for < 30 mA, so that is not an issue.
- On the barrel jack (VIN) 7–12 V is allowed. The onboard regulator (AMS1117) then dissipates (Vin − 5 V) × I: at 12 V and 30 mA that is ~0.2 W — fine. Just don’t combine 12 V with a heavy extra load.
- The **VIN** pin sits in front of the regulator, so you can also feed it from your own 5 V regulator.

4.7 A free second UART on D14/D15

Next to the USB serial port (D0/D1) the Nano has a second hardware UART on **D14 (RX1)** and **D15 (TX1)**. These are free to use: for example to drive an off-the-shelf display, or to control the settings from the transceiver while the USB serial stays free for the laptop.

---

5. Pin overview

Nano pin	Direction	Connected to	Notes
D2	in	microphone PTT (via R2 10 kΩ)	internal pull-up; LOW = pressed
D3	in	push button to GND	internal pull-up; LOW = pressed. Short press =

Nano pin	Direction	Connected to	Notes
			preview, long press (3 s) = chime mode on/off
D8	out	base of Q1 via R1 10 kΩ	HIGH = transmitting
D9	out	C1 → RV1 → microphone AUDIO+	tone, 700–2000 Hz
D13	out	LED (onboard or external via 330 Ω)	lit while transmitting + tail
GND	—	radio GND	via the microphone cable

Do not use: A6, A7 (I2C, no digital I/O on the Nano).

---

6. Wiring it up — step by step

1. **Find the PTT and audio conductors.** Open the manual of your microphone and of your radio. Note the pin numbers of the PTT and audio conductors in the microphone cable *and* in the radio's accessory connector. *(The pinout differs per brand; do not trust internet tables, use the manual of your own model.)*
2. **Measure it.** With a multimeter on continuity: the PTT conductor must make contact with the microphone GND while you press the button.
3. **Cut the microphone cable open** (or solder in a Y-piece) at the PTT conductor and at the audio conductor.
4. **Fit SW1** in the line, as in §3.1. Put the switch next to the radio so you can reach it while transmitting.
5. **Build the PTT output** (§3.2) and connect it to the radio's PTT input.
6. **Build the beep injection** (§3.3) on the microphone's AUDIO+.
7. **Connect the GND** (Nano GND ↔ radio GND).
8. **Connect the power supply**, put the switch in **DIRECT** and first check that the radio transmits normally. Only then switch to **NANO**.

Test measurements

Measurement	Expected
Microphone PTT line, button released	not shorted
Microphone PTT line, button pressed	~0 V (contact to GND)
D2 to GND, button pressed	~0 V (LOW)
D8 to GND, while transmitting	~5 V (HIGH)
Q1 collector to GND, while transmitting	~0 V
AUDIO+ to GND, during the beep	an alternating voltage (measure AC, or listen)

---

7. Software settings

7.1 Directly in the sketch (compile time)

Everything is together at the top of `rogbeep.ino`:

```
#define TXDELAY_DEF   500UL   /* tail after releasing the PTT, 0..1000 ms  */
#define NBEEPS_DEF    3       /* number of tones in the tail, 1..3         */
```

```
#define FREQ1_DEF     1000U   /* tone 1 in Hz, 700..2000                    */
#define FREQ2_DEF     1500U   /* tone 2 in Hz, 700..2000                    */
#define FREQ3_DEF     2000U   /* tone 3 in Hz, 700..2000                    */
#define BEEP_AT_END   0       /* 0 = tones fill the tail, 1 = at the end  */
#define BEEP_WIN_DEF  200UL   /* window used when BEEP_AT_END = 1          */
#define DEBOUNCE_MS   15UL    /* debounce of the PTT input                  */
#define CHIME_CYCLE_MS 1800UL /* length of one chime cycle (ms)             */
#define CHIME_TONES    3       /* tones in the chime (fixed, 1..3)           */
#define LONGPRESS_MS  3000UL  /* hold time for chime mode (ms)              */
```

If you change these values you have to **upload the sketch again** (and press RESET afterwards, see §4.2).

`BEEP_AT_END` is the only setting that changes the character of the beep:

`BEEP_AT_END`	Behaviour
`0` (default, as requested)	The tones fill the entire tail, back to back. With 3 tones each lasts exactly TXDELAY/3.
`1`	Classic roger beep: the tones play in the **last 200 ms** of the tail, right before the transmission ends — still back to back, each 200/3 ms.

7.2 While in use (serial monitor)

Connect the Nano to the PC and open the **Serial monitor** at **115200 baud**. One command per line, terminated with Enter. Commands are case insensitive.

Command	Meaning
`help` / `?`	overview of all commands
`status`	show the current settings
`delay <0-1000>`	TXDELAY in ms
`beeps <1-3>`	number of tones in the tail
`f1 <700-2000>`	frequency of tone 1 (Hz)
`f2 <700-2000>`	frequency of tone 2 (Hz)
`f3 <700-2000>`	frequency of tone 3 (Hz)
`test`	play the tone pattern **without** transmitting
`save`	write the settings to the EEPROM (they survive a power cycle)
`load`	read the settings from the EEPROM
`defaults`	restore the compile-time defaults

Example:

```
delay 700
beeps 3
f1 1200
f2 1800
f3 2000
test
save
```

Errors (value out of range, unknown command) are reported and **not** applied.

> The values in the EEPROM take precedence over the `#define` values at startup. To go back to the “defaults”: `defaults` and then `save`.

`delay` only affects the **tail** after you release the PTT, and `beeps` only affects the **tail** as well. The chime mode always runs on its own cycle of 1800 ms with three tones and is not influenced by either of them.

---

8. Setting the level

1. Put the switch in **DIRECT**, or have the radio on a dummy load / not transmitting.
2. Press the **button** (D3) or send `test` via the serial monitor → the pattern plays, **without** the radio transmitting.
3. Set RV1 to its lowest position and turn it up until the tone is comfortably audible for you (or measure with an AC meter across AUDIO+ and GND).
4. Set the radio's **MIC gain** to a normal value. The beep should not be louder than a raised voice, otherwise the transmitter "pumps".

   The tone is a **square wave** (like almost every roger beep) and switching between two tones may produce a very small click. If you want a softer, more sine-like sound, fit C2 (100 nF) across R3. Check afterwards that the lowest tone (700 Hz) still comes through properly.

---

9. Troubleshooting

Symptom	Likely cause
The Nano does nothing after uploading	You have not pressed the **RESET** button yet (§4.2).
The Nano does not show up in the IDE	Missing CH340 driver (§4.3), or wrong board/processor selected (§4.1).
Upload error "program too large"	Wrong processor selected (Old Bootloader vs. the other one).
Chime mode does not switch on	You did not hold the button long enough (a full 3 s hold; releasing does not count). Hold it a little longer.
Chime mode stays on	Press and hold for **3 seconds** again to switch it off. Check `status` in the serial monitor: it shows "Chime : ON".
The chime sounds shorter/longer than expected	The chime always runs on **1800 ms per cycle** (600 ms per tone with three tones). It is deliberately not linked to TXDELAY; change `CHIME_CYCLE_MS` in the sketch if you want it different.
The chime plays a different number of tones than expected	The chime always plays **three tones** (tone 1, 2 and 3), independent of the `beeps` setting. Change `CHIME_TONES` in the sketch if you want fewer or more.
A short press does nothing	The preview starts when you **release** the button (needed to tell a short from a long press).
The radio does not transmit in the NANO position	Nano is off / resetting; switch wired wrongly; Q1 or R1 not connected; the radio's PTT input is not going low. Check with the LED on D13 and with a multimeter on D8 and on the collector of Q1.
The radio does not transmit in the DIRECT position	The switch or the Y-splice is wrong; check the continuity of the PTT line.

Symptom	Likely cause
No beep	C1, RV1 or R3 open; AUDIO+ connected to the wrong wire; the tone is too quiet (turn RV1 up); or the PTT never arrives so no tail starts. Test with `test` (that works without transmitting).
The beep is too loud / distorted	RV1 turned up too far, or R3 replaced by a lower value; also: the radio's MIC gain is too high.
Hum on the transmitted audio	The 5 V supply is not clean, or the GND connection is long/shared. Use a separate 5 V supply or a battery; fit C3 (100 nF + 10 µF) at the beep injection point; tie the Nano GND to a single point.
The Nano resets while transmitting	The supply cannot deliver enough current (the radio's peak current). Use a supply of at least 1 A, or power the Nano separately from the radio.
The tone sounds harsh	Fit C2 (100 nF) across R3.
The tail feels too short / pressing feels slow	`DEBOUNCE_MS` is set to 15 ms; set it to 5 if you want a faster response.
The settings are gone after `save`	`save` was not executed, or the EEPROM was overwritten by another sketch.

---

10. Possible extensions

A few ideas, from simple to invasive:

1. **Switching automatically.** With a **relay** (5 V, 1 DPDT) the Nano can operate the switch itself: position *DIRECT* when unpowered (NC contact), position *NANO* as soon as the sketch runs. You then never have to touch the switch.
2. **A real "roger beep" mode.** `BEEP_AT_END 1` gives the classic short tone right before the transmission ends — which is what most operators expect.
3. **Two presets.** With an extra switch on a spare pin (e.g. D4 or D5) you can toggle between two presets (e.g. "day profile" short, "contest profile" long).
4. **PTT lockout.** The Nano can also *block* the radio: with `TXDELAY = 0` and a PTT press that is too short, no transmission takes place — handy to prevent "accidents" on the air.
5. **Supply monitor.** With an analog input (A0–A5) you can read the supply voltage and warn via the LED.
6. **Hear the tone without transmitting** (e.g. as a practice tone on the monitor): the button already does this.
7. **Second UART (D14/D15).** Send settings from the transceiver or a display while the USB serial stays free for the laptop.

---

11. Practical notes

- **You may only transmit with a valid amateur radio licence**, and you need a correct antenna and a dummy load for the first tests.
- The tail means the radio keeps transmitting longer: a short press on the PTT now transmits for TXDELAY ms longer. Keep that in mind in a busy net.
- The 15 ms debounce is a compromise: enough to filter out contact bounce, short enough not to notice. The total "delay" in the PTT path is therefore at most ~15 ms — neither audible nor noticeable.

- The beep reaches the transmitter through the **microphone cable**. If your microphone has its own PTT output that interrupts the audio line, check that the tone is still audible.
- With three tones each tone lasts TXDELAY/3. With a small TXDELAY (e.g. 300 ms) that is 100 ms per tone — short, but audible. For a clearly recognisable run of three tones, 600–900 ms TXDELAY is more comfortable.

---

12. The tests (optional)

The logic of the sketch was tested on the host with an Arduino stub, without real hardware:

```
cd test
g++ -std=c++11 -Wall -Wextra -I. -o test_rogbeep test_main.cpp
./test_rogbeep
```

It checks: PTT latency and level, tail length, the tone during the tail, the **exact 1/3 split of three back-to-back tones (every millisecond is scanned; not a single millisecond of silence is allowed between the tones)**, aborting when you press again, TXDELAY = 0, the short press (preview without TX), the **3 s long press (chime mode on, PTT active, a fixed 1800 ms cycle with three tones that ignore both the TXDELAY and the beeps setting, pattern repeats, microphone PTT ignored) and the second long press (everything off again)**, the serial commands including range checks, and saving and reloading the EEPROM.

---

13. Quick reference

```
 1. Upload to the Nano, then press RESET once
 2. Switch to DIRECT  -> check the radio transmits normally
 3. Switch to NANO    -> check the radio transmits
 4. Short button press -> set the beep level with RV1 (no TX)
 5. Hold the button 3 s -> chime mode on (PTT active, 1800 ms cycle, 3 tones)
                         hold 3 s again -> off
 6. Set TXDELAY and tones from the serial monitor
 7. `save`              -> store the settings
```